
Mekholi

Application Architecture & File Reference

Vue 3 + TypeScript ERP frontend for educational institutions (Shikkha ERP)

Stack	Vue 3 (Composition API) · TypeScript · Vite 8 · Pinia · Vue Router 4 · SCSS
Package manager	pnpm
Theme	Single dark theme, no CSS framework (hand-rolled SCSS design system)
Localization	Bilingual — English / Bangla (bn) throughout
Location	D:\Web\ERP\Mekholi

This document gives a high-level map of the codebase: how the pieces fit together, what each folder is responsible for, and what every file currently does. Intended as onboarding material and a reference for future development.

1. Overview

Mekholi is the frontend of an ERP system for educational institutions in Bangladesh (referred to internally as the “Shikkha ERP” — a full functional blueprint for the product lives in `public/Shikkha_ERP_Blueprint.pdf`). The app is a single-page application: a Vue Router-driven shell with a persistent sidebar/header, role-gated pages, and a Pinia store for session state.

There is currently no real backend wired up in development — a dev-only “login as role” flow (`loginAsDev`) lets any of the five user roles be previewed without a server. The HTTP layer is otherwise built to talk to a REST API via `VITE_API_BASE_URL`.

Core stack

Layer	Choice	Notes
Framework	Vue 3	Composition API, <code><script setup></code> throughout
Language	TypeScript	Strict; type-checked via <code>vue-tsc</code>
Build tool	Vite 8	Dev server + production bundling
Package manager	pnpm	<code>pnpm-workspace.yaml</code> present — not <code>npm/yarn</code>
Routing	Vue Router 4	Route table + navigation guards
State	Pinia 2	Setup-store style (see <code>stores/auth.ts</code>)
Styling	SCSS (7-1 architecture)	Single dark theme, design tokens, no UI framework
Linting	oxlint + eslint	Both auto-fix, run together via <code>pnpm lint</code>

User roles

Five roles are defined app-wide and drive route access and (eventually) navigation/menu content: `super_admin`, `institute_admin`, `teacher`, `accountant`, `student_parent_portal`.

2. Request & Auth Flow

- App boot** — `main.ts` creates the Vue app, installs Pinia and the router, imports global SCSS, and mounts to `#app`. `App.vue` is just a `<RouterView />` shell.
- Routing** — `router/routes.ts` defines every route under two layouts: `DefaultLayout` (authenticated app shell) and `AuthLayout` (login). Protected routes carry `beforeEnter: requireAuth`; role-specific routes additionally use `requireRole(...roles)`.
- Guards** — `middleware/auth.ts` redirects to login if not authenticated; `middleware/guest.ts` redirects an already-authenticated user away from login; `middleware/role.ts` redirects to forbidden if the user's role isn't in the allowed list.
- Session state** — `stores/auth.ts` (Pinia) holds user and token. The token is persisted in `localStorage` under `mekholi_auth_token` and rehydrated on refresh. A dev-only path (`loginAsDev`, seeded from `assets/auth/dev_users.json`) fabricates a token like `dev-token-<role>` so any role can be previewed without a backend.
- API calls** — all requests go through `services/http.ts`, a thin fetch wrapper (`get/post/put/delete`) that prefixes `VITE_API_BASE_URL` and auto-attaches `Authorization: Bearer <token>` unless called with `{ auth: false }` (e.g. the login call itself).

composables/useFetch.ts wraps a GET call with reactive data/error/loading state for use inside components.

main.ts → router (guards) → layout (Default/Auth) → page → composable/store → services/http.ts → REST API

3. Folder Structure

```
D:\Web\ERP\Mekholi\
■■■ public/                               Static assets served as-is
■   ■■■ Shikkha_ERP_Blueprint.pdf        Full product spec
■■■ src/
■   ■■■ main.ts                          App entry point
■   ■■■ App.vue                           Root component (RouterView shell)
■   ■■■ assets/
■   ■   ■■■ auth/                         Dev-only seed users
■   ■   ■■■ navigation/                  Role → menu map (JSON)
■   ■   ■■■ styles/main.css              Compiled CSS output (generated)
■   ■■■ components/
■   ■   ■■■ layout/                      AppHeader, AppSidebar, AppFooter
■   ■   ■■■ ui/                          BaseButton, generic reusable pieces
■   ■■■ composables/                     useAuth, useFetch, useToast, useSidebar,
■   ■   ■■■ Institute_Setup/             useAppPreferences, shortcut_key_set,
■   ■                                   useInstituteProfile
■   ■■■ layouts/                         DefaultLayout, AuthLayout
■   ■■■ middleware/                      requireAuth, requireGuest, requireRole
■   ■■■ pages/                           Route-level views
■   ■   ■■■ Institute_Setup/             Feature-scoped views
■   ■■■ router/                          routes.ts, index.ts
■   ■■■ services/                        http.ts (fetch wrapper)
■   ■■■ stores/                          auth.ts (Pinia)
■   ■■■ styles/                          SCSS, 7-1 architecture
■   ■   ■■■ abstracts/                   tokens, mixins, functions
■   ■   ■■■ base/                        reset, base, typography
■   ■   ■■■ components/                  buttons, forms, skeleton
■   ■   ■■■ layout/                      header, sidebar, shell
■   ■   ■■■ pages/                       per-page styles
■   ■   ■■■ themes/                      accents, dark theme, fonts
■   ■■■ types/index.ts                   Shared TypeScript types
■   ■■■ utils/constants.ts               App-wide constants
■■■ package.json
■■■ vite.config.ts
■■■ eslint.config.ts / .oxlintc.json
■■■ SETUP.md / README.md
■■■ CLAUDE.md                            AI-assistant project guide
```

4. File-by-File Reference

Entry & root

File	Purpose
<code>main.ts</code>	Creates the Vue app, installs Pinia + Router, imports global SCSS, mounts to #app.
<code>App.vue</code>	Root component — nothing but a <code><RouterView /></code> .

router/

File	Purpose
<code>routes.ts</code>	The full route table: DefaultLayout routes (home, dashboard, inventory, orders, reports, settings, institute-profile, forbidden) and AuthLayout routes (login), plus a catch-all 404.
<code>index.ts</code>	Creates the router instance (createWebHistory), resets scroll to top on navigation.

middleware/

File	Purpose
<code>auth.ts</code>	requireAuth guard — redirects to login (with redirect query) if not authenticated.
<code>guest.ts</code>	requireGuest guard — redirects an authenticated user away from the login page.
<code>role.ts</code>	requireRole(...roles) — factory returning a guard that redirects to 'forbidden' if the current user's role isn't in the allowed list.

stores/ & services/

File	Purpose
<code>stores/auth.ts</code>	Pinia store (setup style). Holds user + token, exposes login, loginAsDev (dev-only, seeded from dev_users.json), logout, fetchProfile. Persists the token to localStorage and rehydrates the dev user on refresh.
<code>services/http.ts</code>	Thin fetch wrapper: get/post/put/delete. Prefixes VITE_API_BASE_URL, auto-attaches Bearer token unless { auth: false } is passed, throws on non-OK responses.

composables/

File	Purpose
<code>useAuth.ts</code>	Wraps the auth store with storeToRefs — exposes user, isAuthenticated, login/loginAsDev/logout/fetchProfile for components.

File	Purpose
useFetch.ts	Generic GET-and-track composable: reactive data/error/loading + an execute() you can re-call.
useToast.ts	Module-level toast queue (success/error/info), auto-dismiss after a duration.
useSidebar.ts	Module-level singleton state for sidebar collapse + mobile drawer, shared between AppHeader and AppSidebar without prop-drilling.
useAppPreferences.ts	Module-level reactive UI preferences (language, accent theme, Bangla numerals, calendar, density, fonts, per-document language). Persists to localStorage and reflects values onto <html> as data-* attributes so SCSS themes/fonts pick them up globally.
shortcut_key_set.ts	useShortcutKeySet(bindings) — generic keyboard-shortcut binder tied to a component's lifecycle; ignores typing targets unless allowInInputs is set.
Institute_Setup/useInstituteProfile.ts	Module-level reactive Institute Profile & EIIN state (name, EIIN, board, address, logo, etc.), persisted to localStorage, same singleton pattern as useAppPreferences.

layouts/

File	Purpose
DefaultLayout.vue	Authenticated app shell — wraps pages with AppHeader + AppSidebar + AppFooter and the routed page content.
AuthLayout.vue	Minimal layout used for the login page (no sidebar/header).

components/layout/

File	Purpose
AppHeader.vue	Top bar — sidebar toggle, likely user/notification/theme controls.
AppSidebar.vue	Primary navigation — role-aware menu; register new pages here to appear in navigation.
AppFooter.vue	App footer.

components/ui/

File	Purpose
BaseButton.vue	Generic reusable button — base of the hand-rolled UI kit (extend this pattern for future ui/ components rather than adding a component library).

pages/

File	Purpose
HomeView.vue	Landing page after login — feature-card grid (uses the NavItem type).
DashboardView.vue	Main dashboard route.

File	Purpose
InventoryView.vue	Inventory management route.
OrdersView.vue	Orders route.
ReportsView.vue	Reports route.
LanguageThemeSettingsView.vue	UI language, accent theme, calendar, density, font, and per-document language settings — backed by useAppPreferences.
LoginView.vue	Login form + dev role-picker (calls loginAsDev for quick role preview).
ForbiddenView.vue	Shown when requireRole blocks access (403-style page).
NotFoundView.vue	Catch-all 404 page.
Institute_Setup/InstituteProfileView.vue	Institute Profile & EIIN form — backed by useInstituteProfile; captures institution identity, board, type, contact info, logo/letterhead.

types/ & utils/

File	Purpose
types/index.ts	All shared TS types: User/UserRole, ApiResponse<T>, Pagination, NavItem, NavMenu/NavSubMenu/NavigationMap, NotificationItem, AppPreferences and its sub-types (UILanguage, AccentTheme, DateCalendar, fonts, etc.), and InstituteProfile with InstitutionType / EducationBoard.
utils/constants.ts	APP_NAME, API_BASE_URL, TOKEN_KEY, ROLES/ROLE_LABELS, storage keys (SIDEBAR_COLLAPSED_KEY, APP_PREFERENCES_KEY, INSTITUTE_PROFILE_KEY), ACCENT_THEMES swatches, INSTITUTION_TYPES and EDUCATION_BOARDS option lists (each with EN + BN labels).

assets/

File	Purpose
auth/dev_users.json	One demo User per role — powers loginAsDev; dev-only, never for production auth.
navigation/shikkha_erp_navigation.json	Role → menu/sub-menu map (NavigationMap type) driving sidebar content per role.
styles/main.css	Generated CSS output from the SCSS build — gitignored, not hand-edited.

styles/ (SCSS, 7-1 architecture)

File	Purpose
main.scss	Entry point that assembles all partials below into one stylesheet.
abstracts/	_variables.scss (all design tokens: colors, spacing, radius, breakpoints, typography), _mixins.scss, _functions.scss, _index.scss forwarder.

File	Purpose
base/	_reset.scss, _base.scss, _typography.scss — global element defaults.
components/	_buttons.scss, _forms.scss, _skeleton.scss — reusable UI component styles.
layout/	_header.scss, _sidebar.scss, _shell.scss — app-shell structural styles.
pages/	_auth.scss, _language-theme-settings.scss, plus Institute_Setup/ — per-page styles.
themes/	_dark.scss (the single theme), _accents.scss (accent color overrides via [data-accent]), _fonts.scss (font family/size overrides via data-font-* attrs).

5. Key Conventions

- Path alias `@` resolves to `src/` (configured in `vite.config.ts`).
- **State pattern:** anything session-wide and cross-component uses a Pinia store (`stores/auth.ts`); anything that's shared, persisted, reactive-but-not session-critical uses a *module-level singleton* composable instead (`useSidebar`, `useAppPreferences`, `useInstituteProfile`) — same object reused across every call, backed by `localStorage`.
- **Bilingual data:** almost every user-facing option list carries both a `label` and a `label_bn` field (`INSTITUTION_TYPES`, `EDUCATION_BOARDS`, `ACCENT_THEMES`) — follow this shape for any new option list.
- **Preferences → DOM:** UI preferences (language, accent, density, fonts) are reflected as `data-*` attributes on `<html>` so SCSS themes can select on them globally, instead of prop-drilling or scoped classes.
- **New page checklist:** create the view in `pages/` → register the route in `router/routes.ts` (with `requireAuth` / `requireRole` as needed) → add a nav entry in `AppSidebar.vue` if it should be visible in navigation.
- All network calls go through `services/http.ts` — never call `fetch` directly — so auth headers and the API base URL stay centralized.
- No UI/component framework and no light theme are used by design — new UI extends the existing hand-rolled SCSS system in `styles/`.

Available commands

Command	What it does
<code>pnpm dev</code>	Start the Vite dev server (default <code>http://localhost:5173</code>)
<code>pnpm build</code>	Type-check (vue-tsc) then production build to <code>dist/</code>
<code>pnpm type-check</code>	Run vue-tsc only
<code>pnpm lint</code>	Run <code>oxlint</code> then <code>eslint</code> , both with <code>--fix</code>
<code>pnpm scss:build / scss:watch</code>	Optional standalone CSS compile — not needed for normal dev

Generated from the current state of the codebase. For AI-assistant working conventions, see `CLAUDE.md` in the project root; for the full product specification, see `public/Shikkha_ERP_Blueprint.pdf`.